

例如：

$S(2,1)=1$, 两个球 a、b 放在 1 个盒子里, 有 1 种方案, 即 $\{(ab)\}$;

$S(3,1)=1$, 3 个球 a、b、c 放在 1 个盒子里, 有 1 种方案, 即 $\{(abc)\}$;

$S(3,2)=3$, 3 个球 a、b、c 放在两个相同的盒子里, 有 3 种方案, 即 $\{(ab), (c)\}$ 、 $\{(ac), b)\}$ 、 $\{(a), (bc)\}$ 。

【习题】

hdu 4372 “Count the Buildings”, 第一类 Stirling 数。

hdu 2643 “Rank”, 第二类 Stirling 数。

8.4 概率和数学期望

概率和数学期望是概率论和统计学中的数学概念。设有随机变量 X , 出现取值 x_i 的概率是 p_i , 把它们的乘积之和称为数学期望(Expected Value, 或者均值 mean), 记为 $E(X)$:

$$E(X) = \sum_{i=1}^n x_i p_i$$

$E(X)$ 是基本的数学特征之一, 它反映了随机变量平均值的大小。

以妇女的生育率为例, 假设某国有 2000 万个育龄妇女, 不生育妇女有 277 万, 一孩 724 万, 二孩 883 万, 三孩 116 万。记一个妇女的孩子数量是 X , 取值 0、1、2、3, 概率分别是 $277/2000=0.1385$ 、 $724/2000=0.362$ 、 $883/2000=0.4415$ 、 $116/2000=0.058$ 。那么平均每个妇女生育的孩子数量如下:

$$E(X) = 0 \times 0.1385 + 1 \times 0.362 + 2 \times 0.4415 + 3 \times 0.058 = 1.419$$

数学期望具有线性性质。有限个随机变量之和的数学期望等于每个变量的数学期望之和:

$$E(X+Y) = E(X) + E(Y)$$

竞赛中求数学期望的题目一般都会用到它的线性性质。由于线性性质和 DP 的状态转移思想很相似, 所以常常用 DP 来实现。

1. 例题 1

首先看一个简单的例题。

poj 2096 “Collecting Bugs”

一个软件有 s 个子系统, 会产生 n 种 bug。现在要找出所有种类的 bug。假设某人一天发现一个 bug。一个 bug 属于某个子系统的概率是 $1/s$, 属于某种分类的概率是 $1/n$ 。问发现 n 种 bug, 且每个子系统都发现 bug 的天数的期望。 $0 < n, s \leq 1000$ 。

输入: n 和 s ;

输出: 数学期望。

输入样例:

1 2

输出样例:

3.0000

定义状态 $dp[i][j]$, 它表示已经找到 i 种 bug, 并存在于 j 个子系统中, 要达到目标状态还需要的期望天数。其中, $dp[n][s]$ 表示已经找到 n 种 bug, 且存在于 s 个子系统, 说明已经达到了目标, 还需要 0 天, 所以 $dp[n][s] = 0$ 。从 $dp[n][s]$ 倒推回 $dp[0][0]$, 就是本题的答案, 即还没有找到任何 bug 的情况下到达 $dp[n][s]$ 时需要的期望天数。

从 $dp[i][j]$ 开始: 后面 1 天找到 1 个 bug, 可能有以下 4 种情况。

(1) $dp[i][j]$: 发现一个 bug, 属于已经有的 i 个分类和 j 个系统, 概率为 $p1 = (i/n) * (j/s)$ 。这一天相当于浪费了。

(2) $dp[i+1][j]$: 发现一个 bug, 不属于已有分类、属于已有系统, 概率为 $p2 = (1-i/n) * (j/s)$ 。

(3) $dp[i][j+1]$: 发现一个 bug, 属于已有分类、不属于已有系统, 概率为 $p3 = (i/n) * (1-j/s)$ 。

(4) $dp[i+1][j+1]$: 发现一个 bug, 不属于已有系统、不属于已有分类, 概率 $p4 = (1-i/n) * (1-j/s)$ 。

可以验证: $p1 + p2 + p3 + p4 = 1$ 。

状态转移方程如下:

$$dp[i][j] = p1 * dp[i][j] + p2 * dp[i+1][j] + p3 * dp[i][j+1] + p4 * dp[i+1][j+1] + 1 \quad // \text{末尾加上 1 天}$$

整理得到:

$$\begin{aligned} dp[i][j] &= (p2 * dp[i+1][j] + p3 * dp[i][j+1] + p4 * dp[i+1][j+1] + 1) / (1 - p1) \\ &= (n * s + (n-i) * j * dp[i+1][j] + i * (s-j) * dp[i][j+1] + (n-i) * (s-j) * dp[i+1][j+1]) / (n * s - i * j) \end{aligned}$$

在写程序时, 从 $dp[n][s]$ 倒推到 $dp[0][0]$, $dp[0][0]$ 就是答案。

poj 2096 部分程序

```
cin >> n >> s;
for (int i = n; i >= 0; i--)
    for (int j = s; j >= 0; j--) {
        if (i == n && j == s)
            dp[n][s] = 0.0;
        else
            dp[i][j] = (n * s + (n-i) * j * dp[i+1][j] + i * (s-j) * dp[i][j+1]
                + (n-i) * (s-j) * dp[i+1][j+1]) / (n * s - i * j);
    }
```

2. 例题 2

hdu 4035 是经典的迷宫概率问题, 综合了图、数学期望、DP 等内容。

hdu 4035 "Maze"

一个迷宫有 n 个房间, 用 $n-1$ 条隧道连通起来。每个房间里都有陷阱和逃生口。某人的起点在房间 1, 在每个房间都有 3 种可能:

- (1) 落入陷阱被杀死, 回到房间 1, 概率为 k_i ;
- (2) 找到逃生口, 走出迷宫, 概率为 e_i ;
- (3) 在该房间连接的隧道中随机走一条, 进入下一个房间。

求逃出迷宫所要走的隧道数量的期望值。

首先分析这个迷宫, 它是一棵树。

一个有 n 个点、 $n-1$ 条边的无向连通图, 图上肯定没有回路, 这样的图是一棵树。证明如下: 用反证法, 假设有一个回路, 那么在这个回路上可以删除一条边而不影响整体的连通; 删除之后, 还有 n 个点、 $n-2$ 条边, 这是不可能连通的。

要使得有 n 个点的图是连通的, 至少需要 $n-1$ 条边。生成一个连通图, 可以用扩大路径法, 从一个点开始, 每加入一个新的点, 至少需要一条边来连接, 所以 n 个点至少需要 $n-1$ 条边才能连通。

下面是推导过程和编程思路。

1) 定义 DP 状态 $E[i]$

在结点 i 处, 逃出迷宫所要走的边数的期望。

$E[1]$ 就是所求的答案。

根据树的特点, 分析 $E[i]$:

i 是叶子结点, 即 i 没有子结点。在结点 i 有 3 种情况, 即被杀、逃出、回到父结点。

$$E[i] = k_i * E[1] + e_i * 0 + (1 - k_i - e_i) * (E[\text{father}[i]] + 1) \quad (8-1)$$

i 是非叶子结点, 设 i 连接的边数是 m , 有 3 种情况, 即被杀、逃出、转到其他结点。

$$E[i] = k_i * E[1] + e_i * 0 + (1 - k_i - e_i) / m * (E[\text{father}[i]] + 1 + \sum (E[\text{child}[i]] + 1)) \quad (8-2)$$

2) 计算过程

设对于每个结点:

$$E[i] = A_i * E[1] + B_i * E[\text{father}[i]] + C_i$$

目标是求 $E[1]$, $E[1] = A_1 * E[1] + B_1 * 0 + C_1$, 即 $E[1] = C_1 / (1 - A_1)$ 。

在叶子结点上有:

$$A_i = k_i$$

$$B_i = 1 - k_i - e_i$$

$$C_i = 1 - k_i - e_i$$

在非叶子结点上, 设 j 为 i 的子结点, 则:

$$\begin{aligned} \sum (E[\text{child}[i]]) &= \sum E[j] \\ &= \sum (A_j * E[1] + B_j * E[\text{father}[j]] + C_j) \\ &= \sum (A_j * E[1] + B_j * E[i] + C_j) \end{aligned}$$



代入式(8-1)和式(8-2)中,消去 $E[\text{child}[i]]$ 和 $E[\text{father}[j]]$,可以得到非叶子结点的 A_i, B_i, C_i 的表达式。

3) 编程思路

从上面的推导过程可知,计算过程是从叶子结点开始算,再算它们的父结点,直到算出根结点的 A_1, B_1, C_1 ,得到 $E[1]$ 。在编程时,需要按从叶子结点到根结点的顺序遍历每个点。

这个过程用 DFS 编程是最合适的。从根结点 1 出发,用 DFS 遍历整棵树;DFS 到最底层的叶子结点时,计算叶子结点的 A_i, B_i, C_i ,然后逐步回退,再计算非叶子结点的 A_i, B_i, C_i 。

在题目中,图的规模 $n \leq 10\,000$,需要用邻接表存储。请读者在学习第 10 章的相关内容后再回头做这一题。

【习题】

hdu 3853 “LOOPS”,基础题。

hdu 4405 “Aeroplane chess”,简单题。

poj 3071 “Football”,简单概率 DP。

poj 3744 “Scout YYF I”,用矩阵优化求概率。

hdu 4089 “Activation”,2011 年北京区域赛题目,概率 DP,较难。